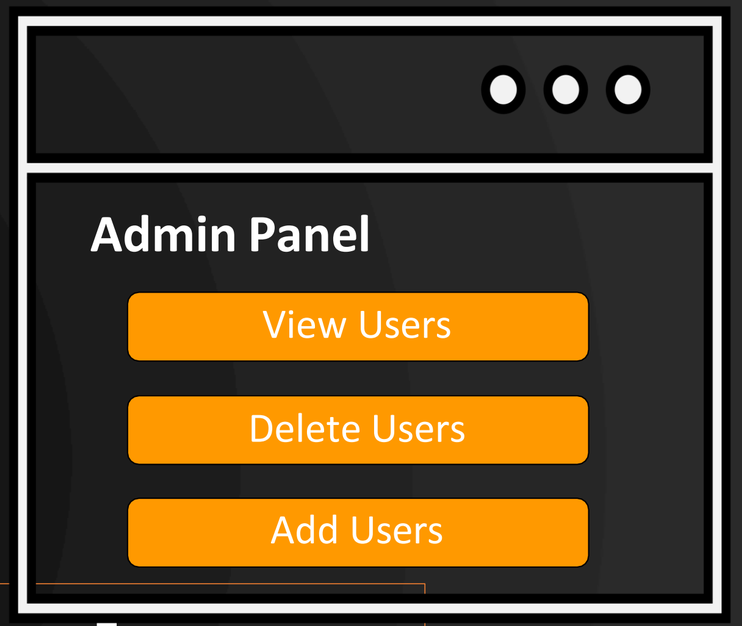
https://vulnerable-website.com/login.jsp?admin=true



Access Control and Broken Access Control

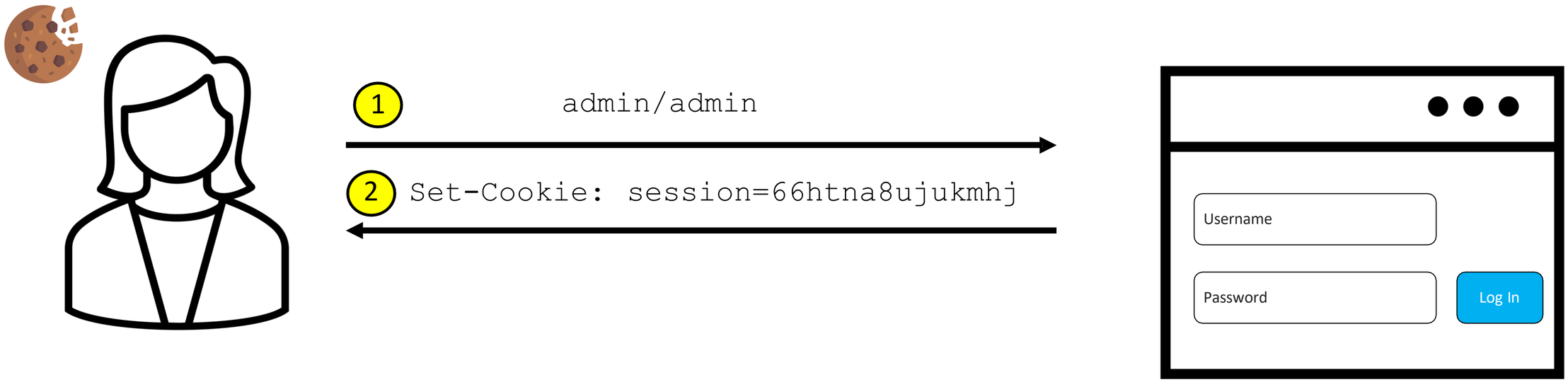
WHAT IS BROKEN ACCESS CONTROL?

Some Terminology...

- Authentication identifies the user and confirms that they say who they say they are.

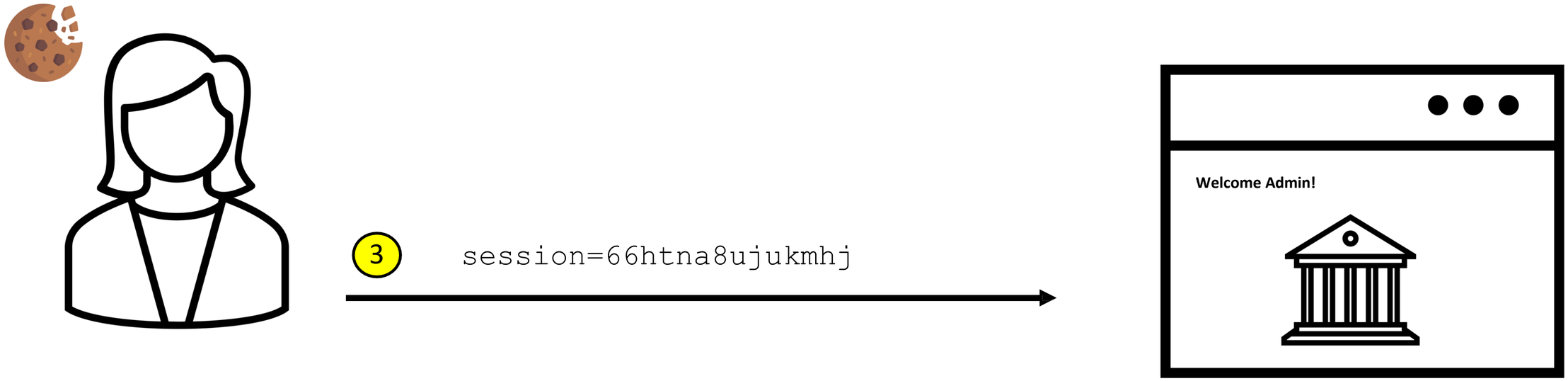
Some Terminology...

- Session Management identifies which subsequent HTTP requests are being made by each user.



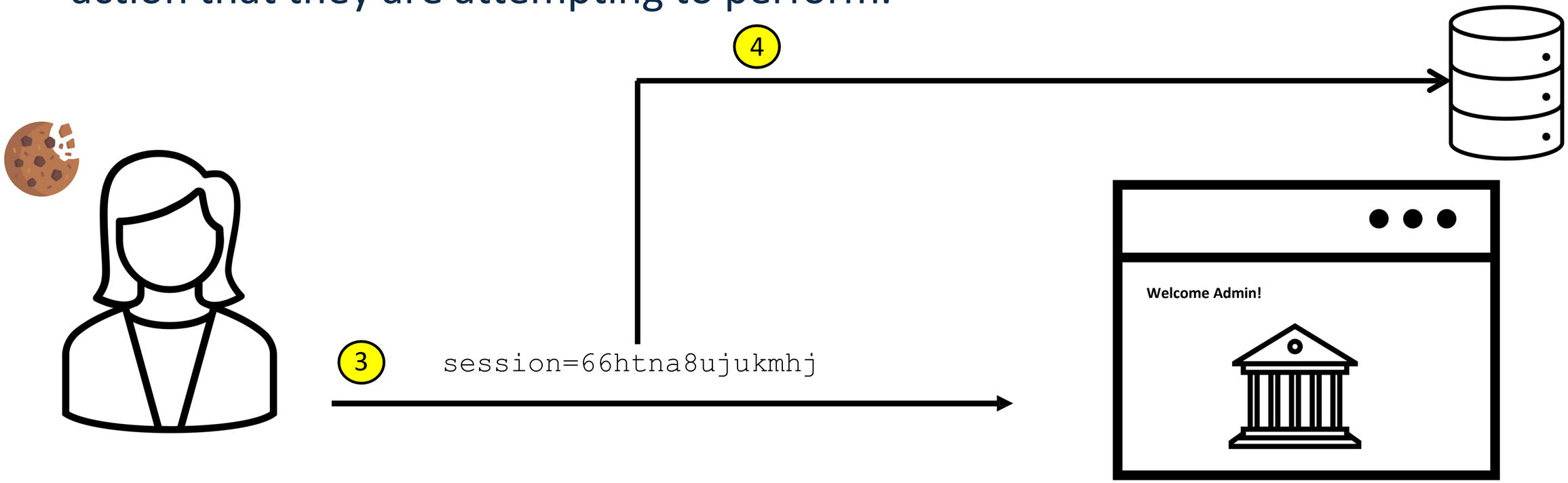
Some Terminology...

- Session Management identifies which subsequent HTTP requests are being made by each user.



Some Terminology...

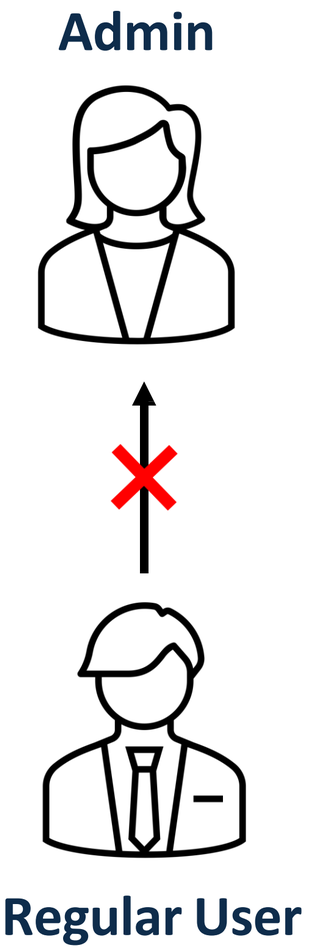
- Access Control determines whether the user is allowed to carry out the action that they are attempting to perform.



Some Terminology...

Three different types of access control:

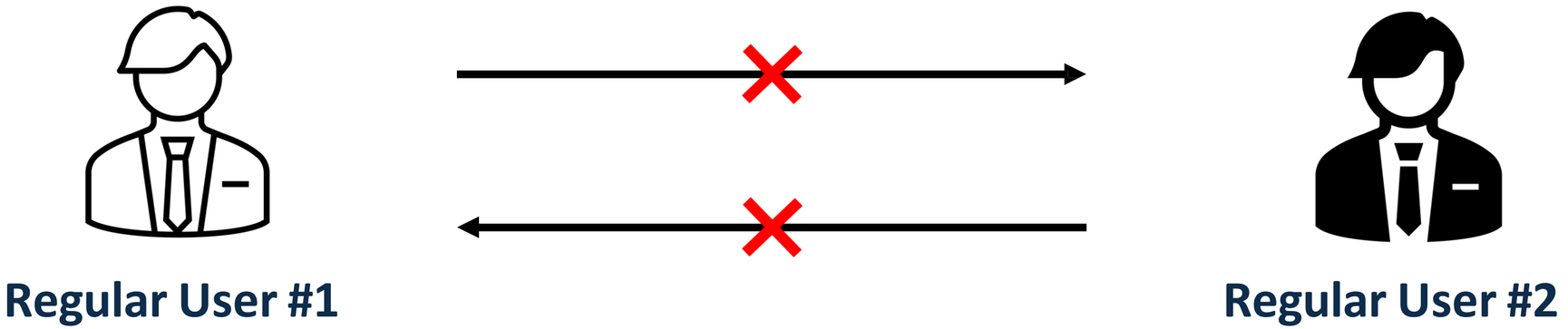
- Vertical Access Control is used to restrict access to functions not available for other users in the organization.



Some Terminology...

Three different types of access control:

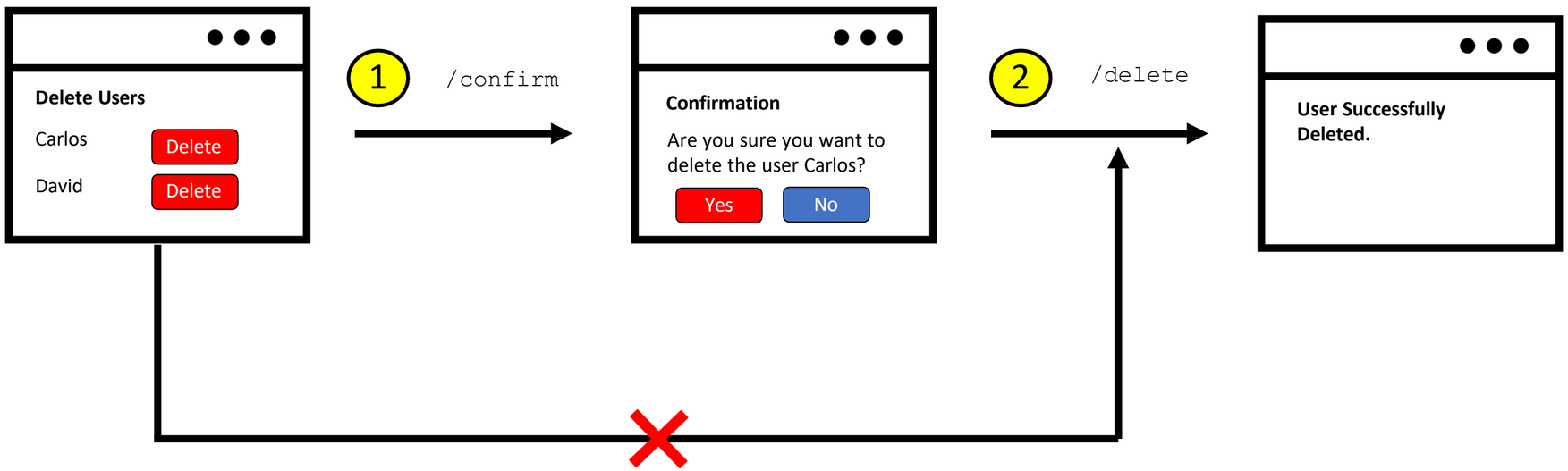
- Horizontal Access Control enables different users to access similar resource types.



Some Terminology...

Three different types of access control:

- Context-Dependent Access Control restricts access to functionality and resources based on the state of the application or the user's interaction with it.



Types of Access Controls

(How they are implemented)

- ❑ Administrative
- ❑ Technical
- ❑ Physical

Types of Access Controls

(Based on what they do)

- ❑ Preventative
- ❑ Detective
- ❑ Corrective
- ❑ Recovery based
- ❑ Directive
- ❑ Compensative

Access Control Models

- ❑ DAC (Discretionary Access Control)
  Owner of a resource and you decide who is access (list of permission)
- ACLs used of this
- ❑ Non-DAC
  Rules and Policies decide by central/predefined
- MAC (Mandatory Access Control)
- RBAC (Role based AC)
- ABAC (Attribute based e.g., locations etc.)
- Risk Based

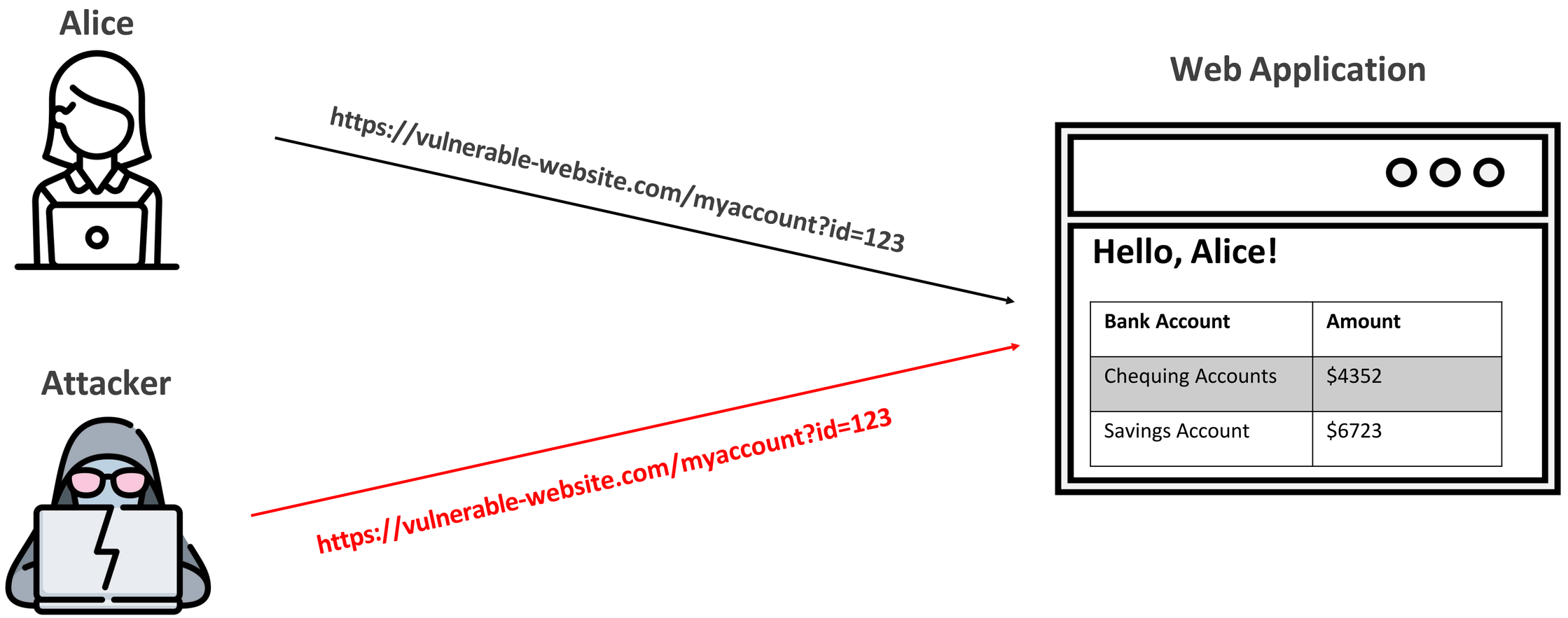
Activity

List out the 10 ACLs that can be used in common applications.

Broken Access Control vulnerabilities arise when users can act outside of their intended permissions. This typically leads to sensitive information disclosure, unauthorized access and modification or destruction of data.

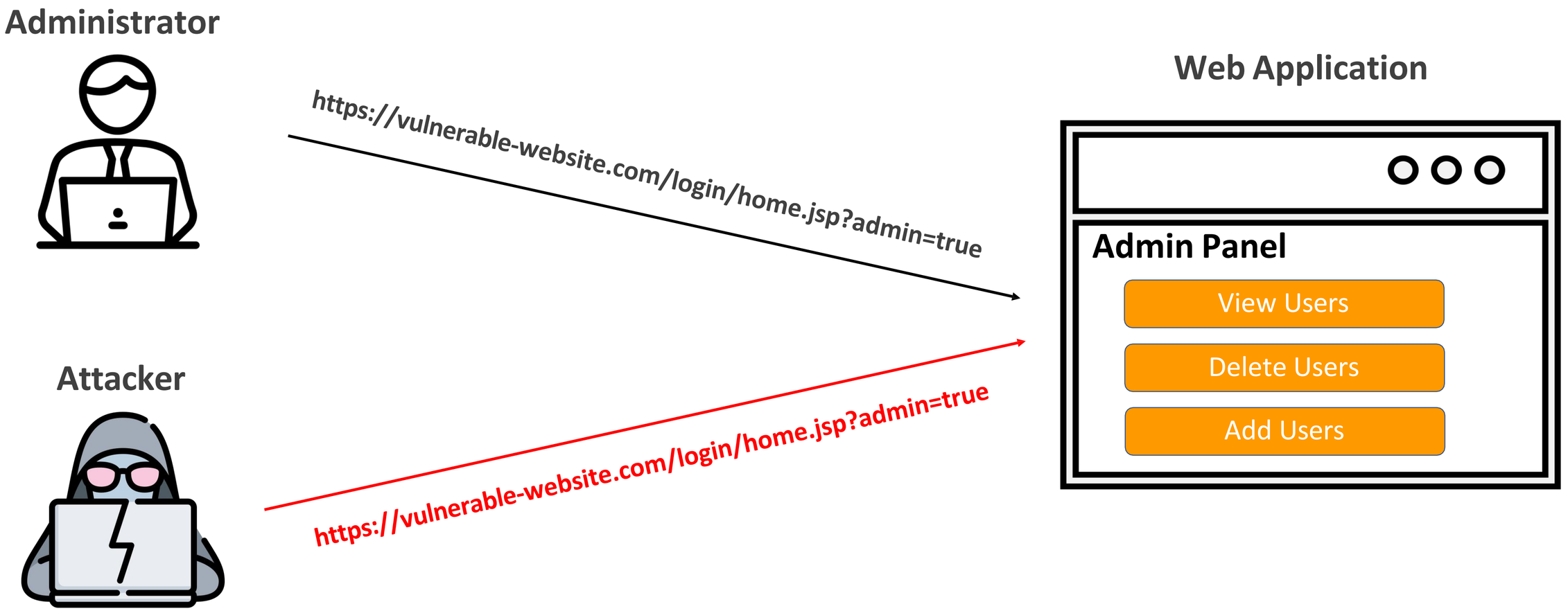
Horizontal Privilege Escalation

Horizontal privilege escalation occurs when an attacker gains access to resources belonging to another user of the same privilege level.



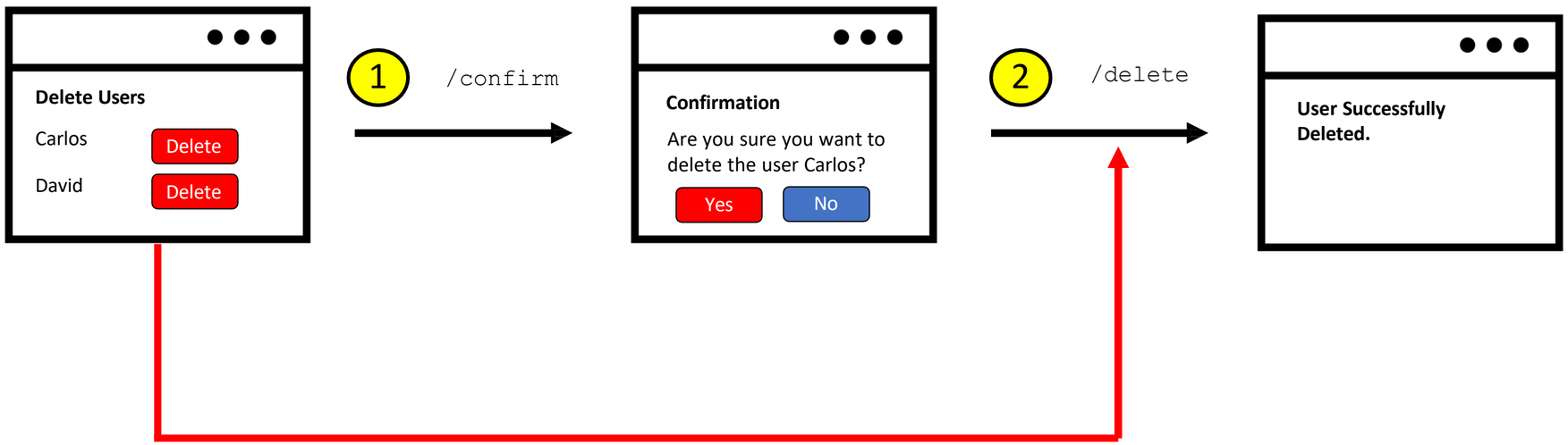
Vertical Privilege Escalation

Vertical privilege escalation occurs when an attacker gains access to privileged functionality that they are not permitted to access.



Access Control Vulnerabilities in Multi-Step Processes

These type of vulnerabilities occur when access control rules are implemented on some of the steps, but ignored on others.



Other Access Control Examples

- Bypassing access control checks by modifying parameters in the URL or HTML page.
- Accessing the API with missing access controls on the POST, PUT and DELETE methods.
- Manipulating metadata, such as replaying or tampering with JSON Web Tokens (JWTs) or a cookie.
- Exploiting CORS misconfiguration that allow API access from unauthorized / untrusted origins.
- Force browsing to authenticated pages as an unauthenticated user.

Spot the Vulnerability?

```
1 public boolean deleteOrder(Long id) {
2    Order order = orderRepository.getOne(id);
3    if (order == null) {
4       log.info("No found order");
5       return false;
6    }
7    User user = order.getUser();
8    orderRepository.delete(order);
9    log.info("Delete order for user {}", user.getId());
10   return true;
11 }
```

Answer: No verification is performed on line 8 to see if the order has been made by the currently logged-in user.

How do you fix this code?

Answer: Ensure the object id of the order belongs to the user making the request.

Code Source: : https://www.securecodewarrior.com/blog/coders-conquer-security-owasp-top-10-api-series-broken-object-level-authorization

Impact of Access Control Vulnerabilities

- Unauthorized access to the application.
  - **C**onfidentiality – Access to other users' data.
  - **I**ntegrity – Access to update other users' data
  - **A**vailability – Access to delete users.
- Can sometimes be chained with other vulnerabilities to gain remote code execution on the host operating system.

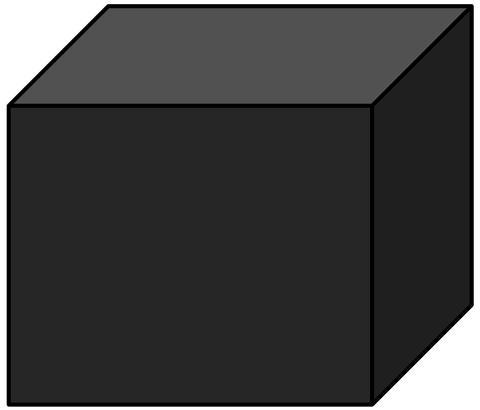
OWASP Top 10

OWASP Top 10 - 2013	OWASP Top 10 - 2017	OWASP Top 10 - 2021
A1 – Injection	A1 – Injection	A1 – Broken Access Control
A2 – Broken Authentication and Session Management	A2 – Broken Authentication	A2 – Cryptographic Failures
A3 – Cross-Site Scripting (XSS)	A3 – Sensitive Data Exposure	A3 - Injection
A4 – Insecure Direct Object References	A4 – XML External Entities (XXE)	A4 – Insecure Design
A5 – Security Misconfiguration	A5 – Broken Access Control	A5 – Security Misconfiguration
A6 – Sensitive Data Exposure	A6 – Security Misconfiguration	A6 – Vulnerable and Outdated Components
A7 – Missing Function Level Access Control	A7 – Cross-Site Scripting (XSS)	A7 – Identification and Authentication Failures
A8 – Cross-Site Request Forgery (CSRF)	A8 – Insecure Deserialization	A8 – Software and Data Integrity Failures
A9 – Using Components with Known Vulnerabilities	A9 – Using Components with Known Vulnerabilities	A9 – Security Logging and Monitoring Failures
A10 – Unvalidated Redirects and Forwards	A10 – Insufficient Logging & Monitoring	A10 – Server-Side Request Forgery (SSRF)

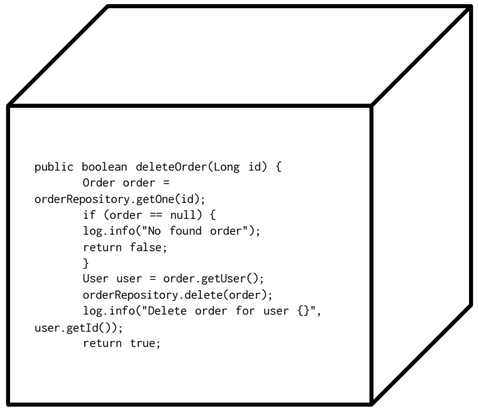
HOW TO FIND ACCESS CONTROL VULNERABILITIES?

Finding Access Control Vulnerabilities

Depends on the perspective of testing.



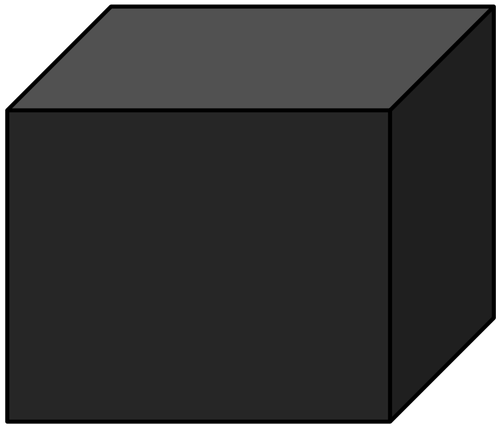
Black Box Testing



White Box Testing

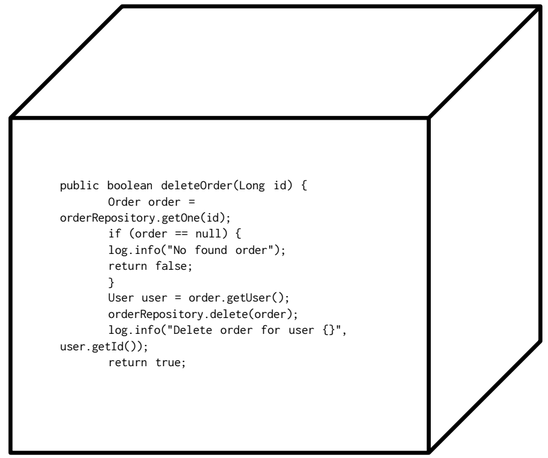
Black-Box Testing

- Map the application.
  - Identify all instances where the web application appears to be interacting with the underlying operating system.
- Understand how access control is implemented for each privilege level.
- Manipulate parameters that are potentially used to make access control decisions in the backend.
- Automate testing using extensions like Autorize.



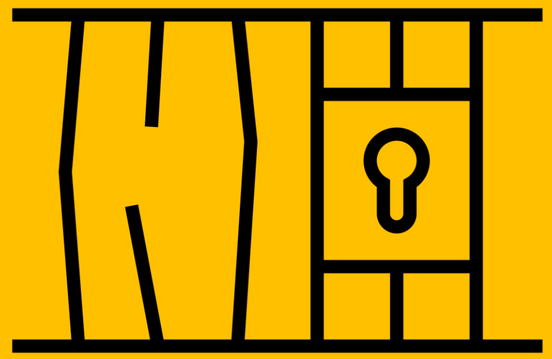
White-Box Testing

- Review the code to identify how access control is implemented in the application.
  - System defaults to open.
  - Weak or missing access control checks on functions / resources.
  - Missing access control rules for POST, PUT and DELETE methods at the API level.
  - Relying solely on client-side input to perform access control decisions.
- Validate potential access control vulnerabilities on a running application.



```
public boolean deleteOrder(Long id) {
        Order order =
orderRepository.getOne(id);
        if (order == null) {
        log.info("No found order");
        return false;
        }
        User user = order.getUser();
        orderRepository.delete(order);
        log.info("Delete order for user {}",
user.getId());
        return true;
```

HOW TO EXPLOIT ACCESS CONTROL VULNERABILITIES?



Exploiting Access Control Vulnerabilities

- Depends on the type of access control vulnerability.
- Usually just a matter of manipulated the vulnerable field / parameter.

LAB APPRENTICE Unprotected admin functionality »

LAB APPRENTICE Unprotected admin functionality with unpredictable URL »

LAB APPRENTICE User role controlled by request parameter »

LAB APPRENTICE User role can be modified in user profile »

LAB APPRENTICE User ID controlled by request parameter »

LAB APPRENTICE User ID controlled by request parameter, with unpredictable user IDs »

LAB APPRENTICE User ID controlled by request parameter with data leakage in redirect »

LAB APPRENTICE User ID controlled by request parameter with password disclosure »

LAB APPRENTICE Insecure direct object references »

LAB PRACTITIONER URL-based access control can be circumvented »

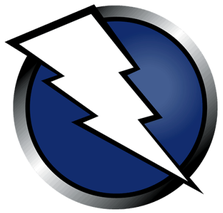
LAB PRACTITIONER Method-based access control can be circumvented »

LAB PRACTITIONER Multi-step process with no access control on one step »

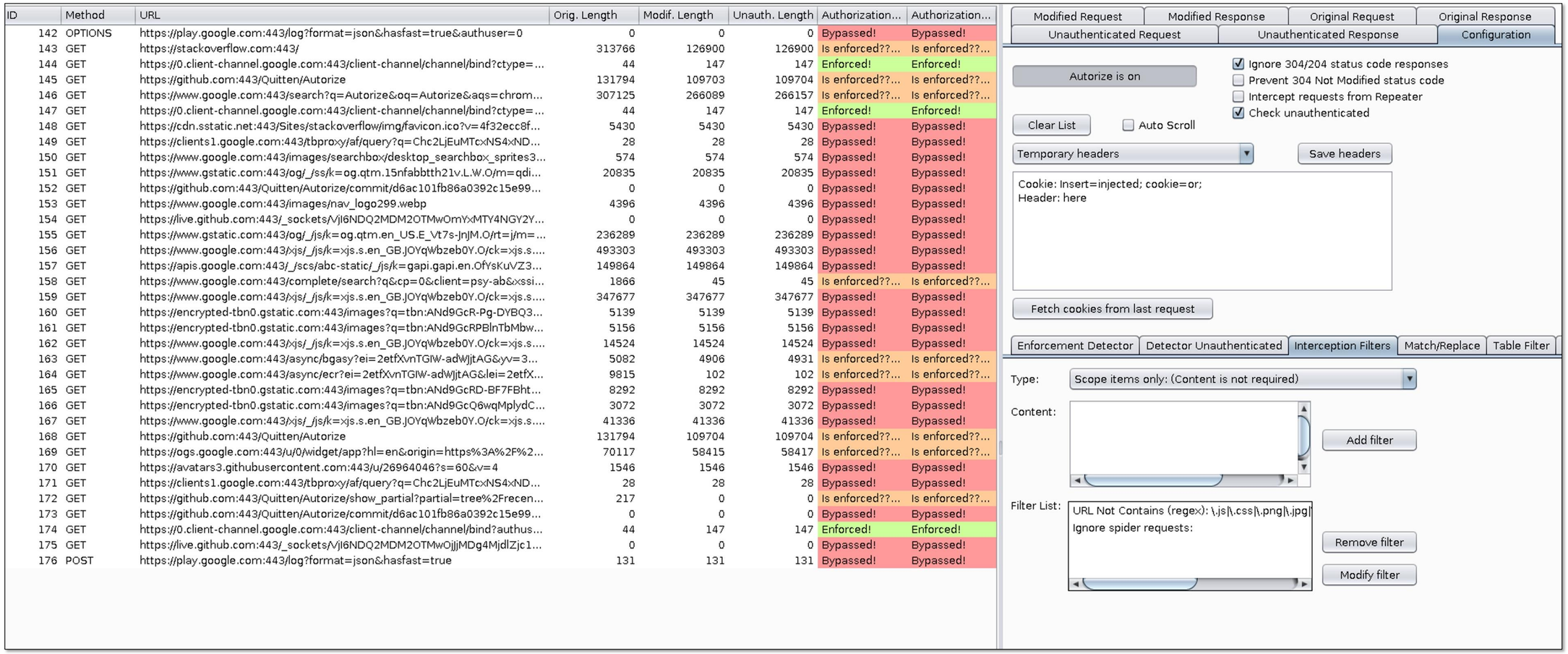
LAB PRACTITIONER Referer-based access control »

Automated Exploitation Tools

Web Application Vulnerability Scanners (WAVS)



Burp Extension - Autorize



ID	Method	URL	Orig. Length	Modif. Length	Unauth. Length	Authorization...	Authorization...
142	OPTIONS	https://play.google.com:443/log?format=json&hasfast=true&authuser=0	0	0	0	Bypassed!	Bypassed!
143	GET	https://stackoverflow.com:443/	313766	126900	126900	Is enforced??...	Is enforced??...
144	GET	https://0.client-channel.google.com:443/client-channel/channel/bind?ctype=...	44	147	147	Enforced!	Enforced!
145	GET	https://github.com:443/Quitten/Autorize	131794	109703	109704	Is enforced??...	Is enforced??...
146	GET	https://www.google.com:443/search?q=Autorize&oq=Autorize&aqs=chrom...	307125	266089	266157	Is enforced??...	Is enforced??...
147	GET	https://0.client-channel.google.com:443/client-channel/channel/bind?ctype=...	44	147	147	Enforced!	Enforced!
148	GET	https://cdn.sstatic.net:443/Sites/stackoverflow/img/favicon.ico?v=4f32ecc8f...	5430	5430	5430	Bypassed!	Bypassed!
149	GET	https://clients1.google.com:443/tbproxy/af/query?q=Chc2LjEuMTcxNS4xND...	28	28	28	Bypassed!	Bypassed!
150	GET	https://www.google.com:443/images/searchbox/desktop_searchbox_sprites3...	574	574	574	Bypassed!	Bypassed!
151	GET	https://www.gstatic.com:443/og/_/ss/k=og.qtm.15nfabbtth21v.L.W.O/m=qdi...	20835	20835	20835	Bypassed!	Bypassed!
152	GET	https://github.com:443/Quitten/Autorize/commit/d6ac101fb86a0392c15e99...	0	0	0	Bypassed!	Bypassed!
153	GET	https://www.google.com:443/images/nav_logo299.webp	4396	4396	4396	Bypassed!	Bypassed!
154	GET	https://live.github.com:443/_sockets/VjI6NDQ2MDM2OTMwOmYxMTY4NGY2Y...	0	0	0	Bypassed!	Bypassed!
155	GET	https://www.gstatic.com:443/og/_/js/k=og.qtm.en_US.E_Vt7s-JnJM.O/rt=j/m=...	236289	236289	236289	Bypassed!	Bypassed!
156	GET	https://www.google.com:443/xjs/_/js/k=xjs.s.en_GB.JOYqWbzeb0Y.O/ck=xjs.s....	493303	493303	493303	Bypassed!	Bypassed!
157	GET	https://apis.google.com:443/_/scs/abc-static/_/js/k=gapi.gapi.en.OfYsKuVZ3...	149864	149864	149864	Bypassed!	Bypassed!
158	GET	https://www.google.com:443/complete/search?q&cp=0&client=psy-ab&xssi...	1866	45	45	Is enforced??...	Is enforced??...
159	GET	https://www.google.com:443/xjs/_/js/k=xjs.s.en_GB.JOYqWbzeb0Y.O/ck=xjs.s....	347677	347677	347677	Bypassed!	Bypassed!
160	GET	https://encrypted-tbn0.gstatic.com:443/images?q=tbn:ANd9GcR-Pg-DYBQ3...	5139	5139	5139	Bypassed!	Bypassed!
161	GET	https://encrypted-tbn0.gstatic.com:443/images?q=tbn:ANd9GcRPBlnTbMbw...	5156	5156	5156	Bypassed!	Bypassed!
162	GET	https://www.google.com:443/xjs/_/js/k=xjs.s.en_GB.JOYqWbzeb0Y.O/ck=xjs.s....	14524	14524	14524	Bypassed!	Bypassed!
163	GET	https://www.google.com:443/async/bgasy?ei=2etfXvnTGIW-adWJjtAG&yv=3...	5082	4906	4931	Is enforced??...	Is enforced??...
164	GET	https://www.google.com:443/async/ecr?ei=2etfXvnTGIW-adWJjtAG&lei=2etfX...	9815	102	102	Is enforced??...	Is enforced??...
165	GET	https://encrypted-tbn0.gstatic.com:443/images?q=tbn:ANd9GcRD-BF7FBht...	8292	8292	8292	Bypassed!	Bypassed!
166	GET	https://encrypted-tbn0.gstatic.com:443/images?q=tbn:ANd9GcQ6wqMplydC...	3072	3072	3072	Bypassed!	Bypassed!
167	GET	https://www.google.com:443/xjs/_/js/k=xjs.s.en_GB.JOYqWbzeb0Y.O/ck=xjs.s....	41336	41336	41336	Bypassed!	Bypassed!
168	GET	https://github.com:443/Quitten/Autorize	131794	109704	109704	Is enforced??...	Is enforced??...
169	GET	https://ogs.google.com:443/u/0/widget/app?hl=en&origin=https%3A%2F%2...	70117	58415	58417	Is enforced??...	Is enforced??...
170	GET	https://avatars3.githubusercontent.com:443/u/26964046?s=60&v=4	1546	1546	1546	Bypassed!	Bypassed!
171	GET	https://clients1.google.com:443/tbproxy/af/query?q=Chc2LjEuMTcxNS4xND...	28	28	28	Bypassed!	Bypassed!
172	GET	https://github.com:443/Quitten/Autorize/show_partial?partial=tree%2Frecen...	217	0	0	Is enforced??...	Is enforced??...
173	GET	https://github.com:443/Quitten/Autorize/commit/d6ac101fb86a0392c15e99...	0	0	0	Bypassed!	Bypassed!
174	GET	https://0.client-channel.google.com:443/client-channel/channel/bind?authus...	44	147	147	Enforced!	Enforced!
175	GET	https://live.github.com:443/_sockets/VjI6NDQ2MDM2OTMwOjJjMDg4MjdlZjc1...	0	0	0	Bypassed!	Bypassed!
176	POST	https://play.google.com:443/log?format=json&hasfast=true	131	131	131	Bypassed!	Bypassed!

Extension Link: : *https://portswigger.net/bappstore/f9bbac8c4acf4aefa4d7dc92a991af2f*

HOW TO PREVENT ACCESS CONTROL VULNERABILITIES?

Preventing Access Control Vulnerabilities

- Use a security-centric design where access is verified first and ensure all requests go through an access control check.
- Except for public resources, deny access by default.
- Apply the principal of least privilege throughout the entire application.
- Consider using attribute or feature-based access control checks instead of role-based access control.
  - NIST Attribute Based Access Control https://www.youtube.com/watch?v=cgTa7YnGfHA&ab_channel=NationalInstituteofStandardsandTechnology
  - NIST Special Publication 800-162: https://nvlpubs.nist.gov/nistpubs/specialpublications/NIST.sp.800-162.pdf
- Access control checks should always be performed on the server side.

Home Work

Spot the Vulnerability?

```
1 public ResponseEntity<?> viewDocument(Long documentId) {
2     Document document = documentRepository.findById(documentId);
3     if (document == null) {
4         log.info("Document not found");
5         return ResponseEntity.notFound().build();
6     }
7     return ResponseEntity.ok(document);
8 }
```

Resources

- Web Security Academy – Access Control
  - *https://portswigger.net/web-security/access-control*
- Web Application Hacker's Handbook
  - *Chapter 8 – Attacking Access Controls*
- OWASP Web Security Testing Guide – Authorization Testing
  - *https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/05-Authorization_Testing/README*
- OWASP Top 10 – A01 Broken Access Control
  - *https://owasp.org/Top10/A01_2021-Broken_Access_Control/*
- OWASP Application Security Verification Standard – V4 Access Control
  - *https://owasp.org/www-pdf-archive/OWASP_Application_Security_Verification_Standard_4.0-en.pdf*